

REACT + VITE REALWORLD EXAMPLE APP

O repositório é um aplicativo em React estruturado utilizando o Vite como "*bundler*" (empacotador) principal. Ele adota uma arquitetura clássica e bem organizada por tipo de responsabilidade, conforme descrito na estrutura de pastas e arquivos abaixo:

1. Raiz do Projeto (Root)

- **cypress/**: Pasta contendo os testes end-to-end automatizados (usando o framework Cypress).
- **dist/**: Essa pasta costuma ser gerada após rodar o build do projeto (onde os arquivos minificados prontos para produção ficam salvos).
- **public/**: Assets e recursos estáticos da aplicação que podem ser acessados publicamente (ex: imagens, ícones root).
- **src/**: O coração do frontend. Quase todo o desenvolvimento é focado dentro deste diretório.
- **package.json** e **pnpm-lock.yaml**: Arquivos que gerenciam as dependências (bibliotecas de terceiros) do ecossistema Node.js.
- **vite.config.js**: Arquivo de configurações do Vite (onde é possível adicionar plugins, definir aliases, config de minificação, entre outros).
- **index.html**: O "ponto de entrada" base do navegador. O Vite injeta diretamente o JavaScript compilado nele via tag `<script type="module" src="/src/main.jsx">`.

2. Diretório Principal (/src)

Dentro da pasta **src/**, o código segue o padrão de separação por **recursos técnicos** ("Feature Folders" / Types):

- **pages/**: Responsável pelos componentes globais de páginas inteiras baseadas nas rotas. Eles orquestram tudo o que é mostrado ao acessar URLs específicas. Exemplos: **Home.jsx**, **Auth.jsx** (Login/Signup), **Article.jsx** (Detalhes), **Profile.jsx**.
- **components/**: Neste diretório moram os componentes "burros" (ou focados em UI) e de exibição parcial que podem ser re-utilizados ao longo do app. Várias páginas importam esses componentes para compor as telas grandes. Exemplos: **Navbar.jsx**, **ArticleList.jsx**, **ArticlePreview.jsx**, **FollowButton.jsx**.
- **hooks/**: Essa pasta agrupa todos os React Hooks customizados que foram abstraídos. No projeto é possível observar arquivos com o prefixo **use...** como **useArticlesQuery**, **useAuth**, **useFavoriteArticleMutation**. Essa abordagem sugere uso intenso de lógicas compartilhadas (utilizando bibliotecas como o **@tanstack/react-query**) para separar muito bem a obtenção de dados/estados complexos dos componentes puramente visuais.
- **models/**: Aqui são mantidos objetos de domínio ou schemas/interfaces da aplicação, ditando o que um "Artigo" ou um "Autor" obrigatoriamente possuem como propriedades (como foi possível observar em **Article.js** e **Author.js**). Ele tem como principal finalidade a padronização e o tratamento dos objetos do formato devolvido pela API de backend.

3. Arquivos Raiz do diretório (/src):

- **main.jsx:** Arquivo responsável por "bootar/montar" a aplicação, usando **ReactDOM.createRoot().render()**.
- **App.jsx:** É o grande "guardião" do layout principal e das definições do React Router (engloba o Header e as sub-rotas).

VUE3 REALWORLD EXAMPLE APP

A estrutura de arquivos do projeto segue um padrão muito moderno, modular e recomendável para aplicações criadas com **Vue 3**, **Vite**, **TypeScript** e **Pinia** (*State Management*). Esta divisão obedece à filosofia "*Separation of Concerns*" (Separação de Preocupações), onde o estado vive globalmente na **store**, lógicas modulares em **composables**, tela em **pages** e requisições cegas em **services**.

Abaixo está detalhado a organização principal:

1. Raiz do Projeto (Root)

- **/dist**: Pasta resultante com a compilação final (Build) do frontend otimizado para produção.
- **/playwright** e **playwright.config.ts**: Configurações e testes End-to-End (E2E) robustos.
- **vite.config.ts**: O arquivo de configuração principal do **Vite**, responsável por empacotar e servir o frontend rapidamente.
- **package.json** e **pnpm-lock.yaml**: Mostra que o ambiente utiliza `pnpm` como gerenciador de dependências, definindo os scripts universais.

2. O Coração do Frontend (/src)

Todo o código-fonte da interface reside dentro de **src**, separado responsavelmente por contexto arquitetural:

- **main.ts**: O ponto de entrada (*entry-point*) da aplicação.
- **App.vue**: O componente raiz (*Root Widget*) onde o roteador e a estrutura-casca são injetados.
- **router.ts**: O arquivo mestre de Roteamento (Vue Router), que mapeia quais URLs carregam quais *pages*.

Pastas vitais de domínio (/src/*):

- **/pages**: São componentes que representam telas inteiras (Views) mapeadas diretamente no roteador. Concentram e coordenam vários componentes menores. Exemplos: **Login.vue**, **Home.vue**, **Profile.vue**.
- **/components**: Componentes 'burros' ou 'semi-burros', isolados metodicamente (botões, modais, listas de postagens, barras de topo) que são reaproveitados pelas *pages*. Exemplos: **AppNavigation.vue**, **ArticlesList.vue**.
- **/composable**: A essência da *Composition* API do Vue 3. Aqui moram funções globais reutilizáveis e abstrações de lógica de tela (como **useArticles()**, **useAuth()**).
- **/store**: Usa a biblioteca **Pinia** (como foi observado nas injeções: **useUserStore()**). Controla dados que devem sobreviver globalmente através das telas, como tokens de acesso, usuário logado, etc.
- **/services**: Arquivos para lidar puramente com chamadas **HTTP** (**fetch** / **axios**). Centraliza domínios de conexão (ex: módulo de login), retendo os tipos de *payload* e de retorno, e separando isso dos componentes visuais.
- **/types** e **/utils**: Tipagens globais do TypeScript (Interfaces de Domínio) e funções matemáticas/formatadoras 'burras' e independentes (ex: manipular uma data ou string).

- **/plugins**: Extensões montadas de forma modular na inicialização do app (i18n, configs terceiras antes do app ser montado).

Característica	REACT + VITE REALWORLD EXAMPLE APP	VUE3 REALWORLD EXAMPLE APP
Framework/Tecnologias	React	Vue 3, Vite, TypeScript e Pinia
Bundler (Empacotador)	Vite	Vite
Filosofia de Arquitetura	Arquitetura clássica organizada por recursos técnicos ("Feature Folders" / Types)	Padrão moderno e modular, seguindo a filosofia " <i>Separation of Concerns</i> " (Separação de Preocupações)
Gerenciamento de Estado	Uso de React Hooks customizados e @tanstack/react-query para obtenção de dados/estados complexos	Biblioteca Pinia (pasta /store) para controle de dados globais
Lógica Reutilizável de Tela	Pasta /hooks (para React Hooks customizados e lógicas compartilhadas)	Pasta /composable (funções globais reutilizáveis e abstrações de lógica de tela, baseadas na <i>Composition</i> API do Vue 3)
Componentes de Páginas/Telas	Pasta /pages (Componentes de páginas inteiras baseadas nas rotas, como Home.jsx)	Pasta /pages (Componentes que representam telas inteiras/Views mapeadas diretamente no roteador, como Login.vue)
Serviços HTTP/API	Lógica de dados integrada em /hooks (e.g., useArticlesQuery).	Pasta /services (Arquivos dedicados para lidar puramente com chamadas HTTP, separando-as dos componentes visuais).

Modelos/Tipagem de Domínio	Pasta /models (Objetos de domínio ou interfaces, como <code>Article.js</code> e <code>Author.js</code>)	Pasta /types (Tipagens globais do TypeScript/Interfaces de Domínio)
Testes End-to-End (E2E)	Framework Cypress (na pasta <code>/cypress</code>)	Framework Playwright (nas pastas <code>/playwright</code> e <code>playwright.config.ts</code>)